A surrogate heuristic for set covering problems

Article *in* European Journal of Operational Research · November 1994

DOI: 10.1016/0377-2217(94)90401-4

2 authors, including:

Luiz A. N. Lorena
National Institute for Space Research
229 PUBLICATIONS **3,651** CITATIONS

European Journal of Operational Research 79 (1994) 138–150
North-Holland

Theory and Methodology

A surrogate heuristic for set covering problems

Luiz Antonio N. Lorena and Fábio Belo Lopes

Laboratório Associado de Computação e Matemática Aplicada (LAC), Instituto Nacional de Pesquisas Espaciais (INPE), Caixa Postal 515, 12201 São José dos Campos SP, Brazil

Received May 1992
Revised September 1992

Abstract: The purpose of this paper is to present a new heuristic for set covering problems, based upon continuous surrogate relaxations and subgradient optimization. The algorithm combines problem reduction tests, an adequate step size control, and avoid preliminary sorting in solving the continuous surrogate relaxations. Computational tests for large scale set covering problems (up to 1 000 rows and 12 000 columns) indicate better-quality results than algorithms based on Lagrangian relaxations in terms of final solutions and mainly in computer times. Although the solving of a single surrogate optimization problem is slower than a corresponding Lagrangian optimization, the overall performance is almost twice as fast. This is due to the smaller number of iterations which is a result of faster convergence and less oscillation.

Keywords: Set covering; Optimization; Surrogate relaxation; Heuristics

1. Introduction

The set covering problems (SCP) considered in this paper can be defined by

$$\begin{aligned} \text{(SCP)} \quad & \text{Minimize} && cx \\ & \text{subject to} && Ax \geqslant e, \\ & && x \in \{0, 1\}^n, \end{aligned}$$

where c is an array of positive components, A is a matrix $(m \times n)$ of zeros and ones, e is an array of ones, and $x_j = 1$ if column j is in the solution, $x_j = 0$ otherwise.

SCP is the problem of covering the rows of A by a subset of the columns at the minimum cost. It is a classical NP-complete problem [20], with applications in facilities location [21,42,52], assembly line balancing [44], information retrieval [13], airline crew scheduling [2,33,34], routing [18] and switching theory [41]. Other application areas are given in [4].

Surveys of research on the set covering problem have been conducted by Christofides and Korman [11], Fisher and Kedia [17] and Garfinkel and Nemhauser [22]. Algorithms have been developed by Balas

Correspondence to: Dr. L.A.N. Lorena, Laboratório Associado de Computação e Matemática Aplicada (LAC), Instituto Nacional de Pesquisas Espaciais (INPE), Caixa Postal 515, 12201 São José dos Campos SP, Brazil.

SSDI 0377-2217(92)00446-5

and Ho [3], Beasley [7,8], Pirkul [38], Salkin and Koncal [43], and Vasko and Wilson [50]. The use of microcomputer in solving optimization problems has been discussed in Bowers [9], Harrison [27], McKay [35], Murdoch [36], Vasko and Wolf [51], Wheller [53], and on an entire issue of *Computers & Operations Research* [25].

The use of heuristics for approximating the solution of SCP is very frequent [3,8,50,51]. Beasley [8] compared favorably his dual heuristic to other ones [3,50,51], mainly in terms of bounds, but with worse computational times.

Subgradient optimization was first introduced by Shor [47] and may be viewed as a generalization of the steepest descent method in which any generalized gradient [10] is substituted for the gradient at a point where the dual function is non-differentiable. But the rules for determining step-sizes must be entirely different. The papers of Polyak [39,40] provide the major treatment about step size selection and convergence results (see also Allen et al. [1]). Various proposals have been offered for the selection of the step sizes (good coverage is contained in Bazaraa and Sherali [6] and Goffin [24]).

The use of subgradient procedures in the context of Lagrangian duality to solve structured combinatorial optimization problems has been shown to be very effective and to work better than classical gradient procedures or column generation techniques (see Held et al. [28] and Nemhauser and Wolsey [37] for an annotated bibliography). On the other hand, due to the poorer structure of a surrogate dual, attempts to determining optimal surrogate multipliers have appeared later, Greenberg [26], Dyer [15], Karwan and Rardin [30] and Sarin et al. [45] have presented several search procedures for computing surrogate dual multipliers by analogy with Lagrangian optimization. Most recently Gavish and Pirkul [23] have applied a new heuristic algorithm for surrogate dual multiplier search in solving the 0–1 multiconstraint knapsack problem. Combining Lagrangian relaxation with a surrogate constraint was successfully explored in John and Kochenberger [29].

In this work we propose a new dual heuristic that presents comparable results in terms of bounds to the dual heuristic of Beasley [8], but with about half the computational times for the same test problems. This feature is due to more stable sequences of intermediate values in the optimization of a surrogate dual of SCP, while getting good feasible solutions for SCP.

In Section 2, the surrogate heuristic for SCP is proposed and its main parts are explained: the solution of the continuous surrogate problems, the construction of intermediate feasible solutions, the tests for removing columns and to stop the algorithm. The heuristic produces stable (low-oscillating) sequences of surrogate values with an adequate choice of step sizes in determined subgradient directions. Section 3 outlines some complementary steps for the implementation of the algorithm and sections four and five are used to present computational tests and conclusions.

2. The Surrogate Heuristic

For any vector $\omega \in \mathbb{R}_+^m$ define the continuous surrogate relaxation $S(\omega)$ of SCP, given by

$$
\begin{array}{lll}
(S(\omega)) & \text{Minimize} & cx \\
 & \text{subject to} & \omega Ax \geqslant \omega e, \\
 & & x \in [0, 1]^n.
\end{array}
$$

The Surrogate Heuristic (SH) uses the relaxation $S(\omega)$, a subgradient direction, construct feasible solutions, and conduct tests for removing columns of matrix A.

The algorithm can be stated as:

Heuristic SH.
INITIALIZE $f_{ub} \leftarrow +\infty$, $f_{lb} \leftarrow -\infty$ and ω by any multiplier such that $\omega \geqslant 0$ and $\omega \neq 0$;
WHILE (stop test = FALSE) DO
Solve $S(\omega)$ and LET the solution be x_ω with optimal
value $v[S(\omega)]$ and j^* the index of the fractional variable;

Set $x_{\omega_{j^*}} = 1$ and construct a feasible solution x_f using
  x_ω and LET $v[F(\omega] \leftarrow \sum_{j=1}^{n} c_j x_{f_j}$;
LET $f_{ub} \leftarrow \min(f_{ub}, v[F(\omega)])$;
LET $f_{lb} \leftarrow \max(f_{lb}, v[S(\omega)])$;
Set $x_{\omega_{j^*}} = 0$ and define ρ for the new step size
  $t_\omega = \rho(f_{ub} - f_{lb})$ and compute
  $\omega_i \leftarrow \max\{0, \omega_i + t_\omega G_i(\omega)/\|G(\omega)\|^2\}$, $i = 1, \ldots, m$,
  with $G(\omega) = e - \sum_{j=1}^{n} a_j x_{\omega_j}$;
Make tests to remove columns;
Perform stopping tests;
END_WHILE;

Heuristic SH is a procedure that approximates the solution for a surrogate dual of the SCP problem. In describing SH, we refer to its main parts: the solution of $S(\omega)$, the construction of a feasible solution, a choice for the step size t_ω, the subgradient direction $G(\omega)$, and the tests for removing columns and stopping the algorithm.

(i) The solution of $S(\omega)$. Heuristic SH solves at each iteration the continuous surrogate relaxation $S(\omega)$, a linear programming problem which has an analytical expression for the optimal solution.

For a given ω, let

$$d_{\omega_j} = \frac{c_j}{\omega a_j}, \quad j = 1, \ldots, n,$$

denote the *efficiency* of the j-th variable, where a_j is the column j of matrix A. If $d_{\omega 1} \leqslant d_{\omega 2} \leqslant \cdots \leqslant d_{\omega n}$, then the optimal solution x_ω of $S(\omega)$ is given by

$$x_\omega = \left(1, 1, 1, \ldots, 1, \frac{\omega e - \sum_{j=1}^{j^*-1} \omega a_j}{\omega a_{j^*}}, 0, 0, 0, 0\right),$$

where $\sum_{j=1}^{j^*-1} \omega a_j \leqslant \omega e \leqslant \sum_{j=1}^{j^*} \omega a_j$ and j^* is the index of the fractional variable (Dantzig [12]; Dudzinski and Walukiewicz [14]). This method of solving $S(\omega)$ is of $O(n \log n)$ in complexity, as the complexity of sorting the *efficiencies* $\{d_{\omega j}\}$. Another method with order $O(n)$ that solves $S(\omega)$ without sorting can be used.

The procedure for solving $S(\omega)$ without sorting is similar to that described in Balas and Zemel [5], except for the threshold, i.e. the problem size under which the usual solution method (based on sorting the variable *efficiencies*) becomes preferable to the procedure without sorting. The threshold used is zero, i.e. the extreme case where the solution of $S(\omega)$ is reached through the procedure without sorting (see also the procedure NKR of Fayard and Plateau [16]).

The procedure performs a dichotomous search in the set $\{d_{\omega j}: j = 1, \ldots, n\}$. An initial index is given and at the end of the procedure the index j^* for that intermediate ω in SH is obtained. Then set

$$x_{\omega j} = \begin{cases} 1 & \forall j < j^*, \\ 0 & \forall j > j^* \end{cases} \quad \text{and} \quad x_{\omega j^*} = \frac{\omega e - \sum_{j=1}^{j^*-1} \omega a_j}{\omega a_{j^*}}.$$

It seems to be computationally more efficient to choose the initial index to be that one found in the previous iteration of SH. This proposal is supported by the results obtained with the application of the procedure on 7 set of problems (to be presented in Section 4), with a gain of 47% in terms of computational time. A comparison was also made with an efficient sorting procedure, the classical heap-sort [49], with 135% of gain.

(ii) The construction of a feasible solution. Assume that the columns are ordered in increasing cost order, and the columns with equal cost are ordered in decreasing order of number of rows that they cover.

The procedure used for construction of a feasible solution x_f using x_ω is similar to that one in Beasley [8]. Set $x_{\omega j*} = 1$ and for each row i which is uncovered ($\sum_{j=1}^{n} a_{ij} x_{\omega j} = 0$) set $x_{\omega k} = 1$, where k is the column corresponding to $\min[j \mid a_{ij} = 1,\ j = 1, \ldots, n]$. After that, try to reduce the redundancies.

(iii) The subgradient direction $G(\omega)$ and the step size t_ω. Setting $x_{\omega j*} = 0$, and for $u = d_{\omega j*} \cdot \omega = (c_{j*}/\omega a_{j*}) \cdot \omega$, the direction

$$G(\omega) = e - \sum_{j=1}^{n} a_j x_{\omega j}$$

is a subgradient for the Lagrangian function

$$(L(u)) \quad \text{Minimize} \quad cx + u(e - Ax)$$
$$\text{subject to} \quad x \in \{0, 1\}^n.$$

Is also immediate that $v[S(\omega)] = v[(L(u)]$, where $v[(\cdot)]$ indicates the optimal value of problem $(\cdot)$ (Lorena and Plateau [32]). As a consequence it can be conjectured that SH is also a Lagrangian heuristic, but as the index $j*$ and the parameter $d_{\omega j*} = c_{j*}/(\omega a_{j*})$ result in the solution of $S(\omega)$, it is impossible to work directly with Lagrangians.

It is well known that slow convergence and non-monotonic behavior are two main undesirable features of subgradient methods. Alternative procedures have been proposed to get a monotonic behavior using elaborated subgradient like algorithms [19,31,46,48]. To prevent these problems, an easily computational expression, suggested in Lorena and Plateau [32], can be used for the step size

$$t_\omega = \rho \cdot (f_{ub} - f_{lb}), \quad \text{setting } \rho = \alpha / d_{\omega j*}.$$

Empirically, the parameter α is initialized to obtain $t_\omega / \| G(\omega) \|^2 = 20$, in the first iteration. If f_{lb} has not been increased in the last 6 iterations then set $\alpha = 0.90\alpha$. In some instances the value of the step size can be too large at the initial iterations, and a superior limit of 20 is imposed to $t_\omega / \| G(\omega) \|^2$.

These choices produce nice results when applied to the test problems presented in Section 4. Figures 1 and 2 represent sequences of feasible solutions and of values for $v(S(\cdot))$ and $v(L(\cdot))$ versus iterations,

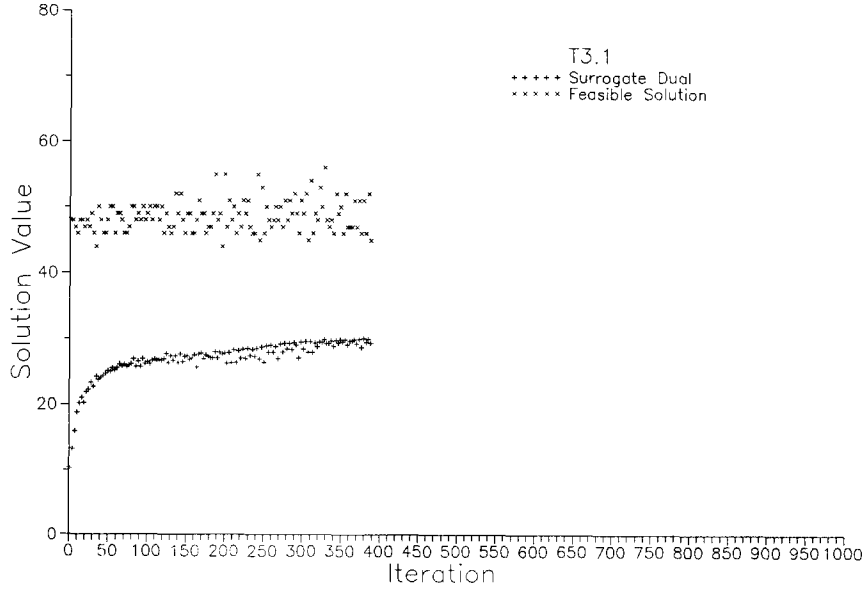

Figure 1

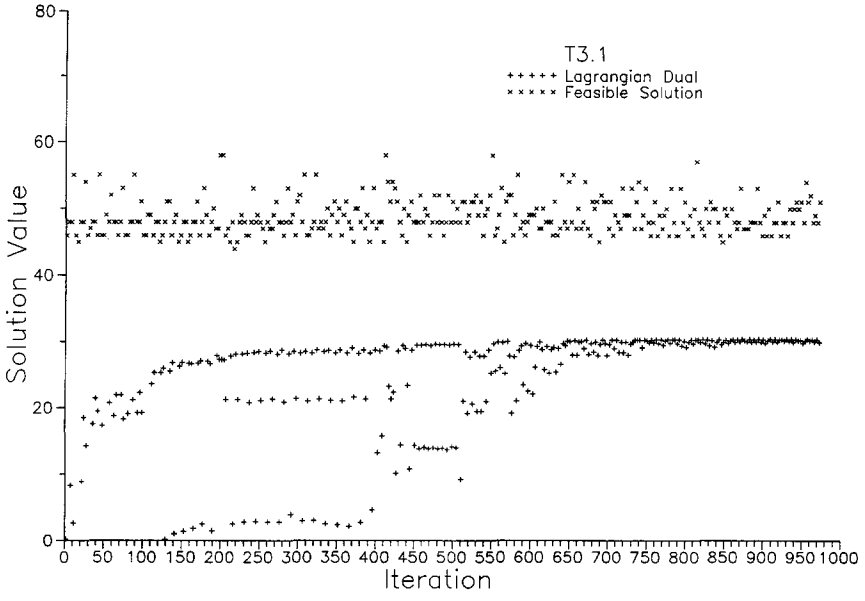

Figure 2

using heuristic SH and an implementation of the Lagrangian heuristic (Beasley [8]). The test problem used, named T3.1, is of size 1000×12000 (see Table 1). The sequence $\{v[S(\cdot)]\}$ is very stable when compared with sequence $\{v[L(\cdot)]\}$ and SH stops early with a good feasible solution. These characteristics are conserved by a lot of different values for α. The principal part of the step size definition is the use of $d_{\omega j*}$ approximately reproduces the traditional subgradient methods for $L(u)$, while avoiding oscillating behavior and slow convergence.

(iv) Removing columns. If $v[S(\omega) \mid x_{\omega j} = 1 - \gamma] > f_{ub}$, a variable $x_{\omega j}$ can be definitively fixed at γ ($\gamma \in \{0, 1\}$). Due the low-oscillating characteristic of sequence $v(S(\omega))$, this test remove columns, even in early steps of heuristic SH, improving computational times. The test is only applied when f_{ub} or f_{lb} has been improved in the last algorithm iteration.

(v) The stopping tests. The algorithm stops when one of the following conditions is satisfied:

(a) The number of iterations is greater than 1000; or

(b) $f_{ub} - f_{lb} < 1$, f_{ub} is an optimal solution for SCP; or

(c) the value f_{lb} has not increased more than 1% in the last 10 computed values. These values are collected with a frequency of 30 consecutive subgradient iterations.

3. Complementary steps

For the implementation of heuristic SH some complementary steps were considered. An initial reduction of the SCP, an efficient data structure and a final heuristic.

3.1. Problem reduction

A number of reduction tests are well known in the literature (see Beasley [7]). Here we use some of those found to be more effective, beyond the test described in (iv), Section 2. These tests are conducted before the heuristic SH.

(i) Column domination. Any column j whose rows $\{i \mid a_{ij} = 1, i = 1, \ldots, m\}$ can be covered by other columns for a cost less than c_j can be deleted from the problem. Assuming the initial ordination of columns (see Section 2, (ii)), let

$$\beta_i = \min\{j | a_{ij} = 1, j = 1, \ldots, n\}, \quad i = 1, \ldots, m,$$

and

$$H_j = \bigcup_{i=1}^{m} \{\beta_i | a_{ij} = 1\}, \quad j = 1, \ldots, n.$$

Then if $\Sigma_{k \in H_j} c_k < c_j$, the column j can be deleted from the problem.

This procedure differs from the one suggested by Beasley [7] since it avoids to add the cost of a column more than once time. It is therefore about 20% more efficient in terms of column reduction, with a very low overhead in computational time.

(ii) Column inclusion. If a row i, $i = 1, \ldots, m$, is covered by only one column, this column must be at the optimal solution. Note that when a column is set to be at the optimal solution, all the rows (constraints) that are covered by this column are automatically satisfied and can be deleted from the problem.

(iii) Null column reduction. Applying the tests above may result in columns which cover no rows, i.e. they are null columns. Those columns must be deleted from the problem.

3.2. Data structure

An efficient data structure was developed in order to apply heuristic SH to large scale set covering problems on an IBM PC based microcomputer. Problems up to 1000 rows and 12000 columns were solved in acceptable computation time (about 750 seconds). The total memory used by the data is about ($25n + 12m + 2$ times the number of ones in matrix A) bytes, where n is the number of columns and m is the number of rows of matrix A.

3.3. Getting a better feasible solution

A Replacement Heuristic (RH) was considered to be applied after heuristic SH as an attempt to get better feasible solutions to SCP. The heuristic RH uses x_f and f_{ub}, the final feasible solution of SH and the cost of x_f, respectively.

Heuristic RH.

Step 1. For each variable fixed to one in x_f, fix the variable equal to zero and repeat the step to obtain a feasible solution (Section 2, (ii)) of SH.

Step 2. A number of different feasible solutions are defined in Step 1. Take the best of these solutions and denote it by x_b with cost f_b. If x_b is better than x_f ($f_b < f_{ub}$) then set $x_f \leftarrow x_b$ and $f_{ub} \leftarrow f_b$, and return to Step 1. If $f_b \geqslant f_{ub}$ then stop.

4. Computational tests

The computational tests were aimed at obtaining better-quality solutions to large scale set covering problems using microcomputers, and to compare the implementation of SH with a Lagrangian based heuristic.

In order to compare heuristic SH with the Lagrangian heuristic (LH) of Beasley [8], the same data structure was used for both algorithms, but with the following slight modifications in LH:

(a) using $f_{ub} - f_{lb} < 1$ instead of the original stop test $f_{ub} = f_{lb}$; and

(b) considering the heuristic RH at the end of the algorithm.

The Lagrangian heuristic with these adaptations was denoted Modified Lagrangian Heuristic (MLH).

The test problems were taken from Balas and Ho [3] (problem sets 4, 5 and 6) and Beasley [7] (problem sets A, B, C and D). The other problem sets (T_1,T_2 and T_3) were randomly generated using the scheme of Balas and Ho [3], i.e. every column cost has a range of 1 to 100, every column covers at least

Table 1
Summary of the problem sets

Problem set	Number of rows before reduction	Number of columns before reduction	Density (%) before reduction	Number of rows after reduction [a]	Number of columns after reduction [a]	Density (%) after reduction [a]	Number of problems in problem set
4	200	1000	2	182	202	2.4	10
5	200	2000	2	182	218	2.4	10
6	200	1000	5	200	237	5.3	5
A	300	3000	2	300	394	2.3	5
B	300	3000	5	300	490	5.2	5
C	400	4000	2	400	561	2.2	5
D	400	4000	5	400	697	5.1	5
T1	1000	5000	5	1000	1238	6.1	5
T2	1000	8000	5	1000	1730	6.1	5
T3	1000	12000	5	1000	2489	6.0	5

[a] Average value for each problem set.

one row, and every row is covered by at least two columns. Table 1 presents a summary of these test problems before and after an initial problem reduction program. This program implements the problem reduction tests of Section 3.1. In the following when a test problem is referred, it concerns in fact a reduced test problem.

The algorithms were coded in C, running on an IBM PC-386 based computer with 1 Mb of random access memory, 40 Mb of hard disk, super VGA color display and a numeric co-processor.

The behavior of sequences $\{v[S(\cdot)]\}$ and $\{v[L(\cdot)]\}$ showed in Figures 1 and 2 for problem T3.1 was also observed by all the other test problems. Tables 2–4 give the computational results for each problem considered: problem sets 4–6 in Table 2, problem sets A–D in Table 3, and problem sets T1–T3 in Table 4. The optimal solution is known for the problem sets 4–6 and A–D.

Some comments about Tables 2–4:

(a) The results for the Lagrangian heuristic LH are taken directly from Beasley [8]. They are in the tables only for comparisons.

(b) The column 'Iteration number' of f_{ub} shows the iteration number when the best feasible solution was found. Heuristic SH found their best feasible solutions before the modified Lagrangian heuristic MLH in all but 9 of the 60 test problems.

(c) The execution times include only those used in the execution of the main program (MLH or SH) and replacement heuristic RH (when necessary). Time for the initial reduction and input/output are not considered. RH takes about 2% of the total execution time.

(d) RH reaches the optimal solution in 8 problems (2 in MLH and 6 in SH), what seems to be nice if its low overhead in time is taken in consideration. Furthermore, RH makes better 13 feasible solutions in MLH and 21 in SH.

Table 5 shows the average deviation from optimal ($100 \times$ [solution value $-$ optimal]/optimal) using LH, MLH and SH. The three heuristics have almost the same percentage deviation from the optimal, but SH found 27 optimal solutions, while LHM found 24 and LH only 19.

The most important difference was felt in terms of number of subgradient iterations and computational times. Tables 6 and 7 show the average number of iterations and the average computational times, respectively, for each problem set. In average MLH runs each subgradient iteration faster than SH, but when the total computational time is considered, SH is almost twice as fast. This is due to the smaller number of iterations, which is a result of faster convergence and low oscillation. Increasing the number of variables makes the number of iterations for SH be almost linear. This sounds interesting for extremely large scale problems such as those of crew scheduling.

Table 2
Computational results, problem sets 4, 5, 6

		LH				MLH							SH						
Problem	Optimal solution	Best feasible solution	Dual solution	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left
4.1	429	429	428.98	581	79	429	446	–	428.18	12.20	446	104	429	197	–	428.84	8.85	197	84
4.2	512	512	512	469	60	512	253	–	511.03	13.52	358	124	512	123	–	511.04	7.47	123	104
4.3	516	516	516	331	64	516	140	–	515.02	11.21	311	115	516	80	–	515.11	5.71	81	94
4.4	494	495	493.93	640	108	495	106	495	493.93	23.46	729	118	495	81	495	493.73	19.61	390	113
4.5	512	512	512	332	60	512	142	–	511.27	8.35	246	107	512	75	–	511.75	4.95	75	84
4.6	560	561	557.17	761	143	560	191	560	557.15	25.93	717	139	560	128	560	556.74	22.91	390	139
4.7	430	430	429.97	567	70	430	332	–	429.68	10.38	332	143	430	177	–	429.43	8.85	177	88
4.8	492	493	488.61	643	131	492	535	492	488.60	20.16	577	123	495	174	493	488.20	23.24	390	131
4.9	641	641	638.29	828	136	641	626	641	638.32	34.89	913	143	641	382	641	638.04	25.27	390	136
4.10	514	514	513.42	591	87	514	314	–	513.05	11.70	423	91	514	88	–	513.01	5.55	116	84
5.1	253	255	251.16	691	180	255	203	255	251.16	32.08	806	180	254	125	253	251.04	25.82	420	139
5.2	302	304	299.62	745	206	306	547	306	299.61	31.97	731	224	305	342	302	299.19	30.00	420	171
5.3	226	226	225.98	467	64	226	209	–	225.10	8.08	209	152	226	70	–	225.52	4.45	71	85
5.4	242	242	240.48	526	132	242	90	242	240.49	21.64	583	152	242	153	242	240.46	24.61	420	131
5.5	211	211	210.96	424	72	211	213	–	210.10	7.86	273	162	211	51	–	210.10	3.96	79	98
5.6	213	213	212.48	536	91	213	133	–	212.04	8.30	246	128	213	84	–	212.15	6.37	113	100
5.7	293	294	291.74	763	147	293	167	293	291.73	27.41	821	140	294	129	293	291.66	25.05	450	131
5.8	288	288	286.88	839	139	288	120	288	286.88	32.08	858	153	288	194	288	286.67	27.19	450	139
5.9	279	279	278.98	777	83	279	167	–	278.00	11.59	308	132	279	77	–	278.03	13.30	244	115
5.10	265	265	264.97	453	68	265	168	–	264.04	6.65	179	133	265	60	–	264.23	3.96	61	95
6.1	138	141	133.09	872	158	141	592	141	133.09	47.03	830	164	140	242	138	132.97	28.46	420	131
6.2	146	146	140.36	779	130	147	517	147	140.34	42.74	743	148	150	19	149	140.20	31.37	450	159
6.3	145	145	139.85	883	119	145	139	145	139.89	45.54	898	130	145	146	145	139.82	25.98	420	119
6.4	131	141	128.89	714	97	131	180	131	128.89	29.34	672	104	131	29	131	128.70	19.50	360	99
6.5	161	162	153.01	631	159	163	830	163	153.18	55.82	954	172	162	387	161	153.07	34.78	480	147

Table 3
Computational results, problem sets A, B, C, D

		LH				MLH							SH						
Problem	Optimal solution	Best feasible solution	Dual solution	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left
A.1	253	255	246.63	753	373	256	198	256	246.64	59.17	740	345	258	13	254	245.98	45.10	390	325
A.2	252	256	247.25	826	380	256	436	256	247.25	57.96	719	349	255	300	255	247.17	49.12	450	311
A.3	232	234	227.80	601	331	235	682	235	227.84	62.80	792	334	239	272	234	227.19	42.74	360	320
A.4	234	235	231.12	859	244	234	447	234	231.15	57.80	821	241	234	185	234	231.05	39.72	420	218
A.5	236	237	234.85	636	250	237	291	237	234.86	54.45	827	228	238	76	238	234.79	36.70	390	233
B.1	69	70	64.38	811	251	70	249	70	64.42	129.61	1131	279	70	237	70	64.28	50.16	390	253
B.2	76	77	69.21	850	332	78	721	76	69.23	151.42	1042	330	76	69	76	69.15	66.37	480	302
B.3	80	80	74.08	720	264	81	194	81	74.07	97.03	753	317	81	18	81	74.04	59.89	450	294
B.4	79	80	71.11	780	363	82	570	81	71.10	115.49	767	420	81	159	81	71.05	75.82	480	382
B.5	72	72	67.60	845	212	72	179	72	67.60	91.15	833	240	72	192	72	67.52	48.40	390	212
C.1	227	230	223.70	835	401	230	497	229	223.69	92.25	765	389	230	118	227	223.52	67.25	420	290
C.2	219	223	212.71	1021	577	222	465	222	212.68	121.53	955	505	226	128	222	212.22	68.62	390	500
C.3	243	251	234.42	968	788	249	774	245	234.38	113.73	816	535	251	273	251	234.08	81.97	480	588
C.4	219	224	213.63	929	574	225	688	221	213.64	126.04	947	463	229	212	224	213.52	89.39	510	504
C.5	215	217	211.44	798	378	217	209	216	211.44	103.62	903	369	217	163	216	211.27	68.29	450	338
D.1	60	61	55.15	858	345	61	48	61	55.24	213.18	1171	379	60	272	60	55.11	79.94	420	313
D.2	66	68	59.15	866	449	68	231	66	59.12	146.48	647	412	68	252	68	59.14	112.25	480	450
D.3	72	75	64.97	952	495	73	329	73	64.96	210.27	957	456	75	47	75	64.75	97.47	390	503
D.4	62	64	55.66	649	437	63	236	63	55.71	188.62	929	434	63	34	63	55.42	80.27	360	408
D.5	61	62	58.56	935	227	61	567	61	58.58	175.00	1170	226	63	185	62	58.44	74.06	420	234

Table 4
Computational results, problem sets T1, T2, T3

		LH				MLH							SH						
Problem	Optimal solution	Best feasible solution	Dual solution	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left	Best feasible solution (f_{ub})	Iteration number of (f_{ub})	Feasible solution after (RH)	Dual solution	Time (sec.)	Number of iterations	Number of columns left
T1.1	–	–	–	–	–	69	585	69	57.9	615.66	1032	655	71	22	69	57.67	372.30	450	636
T1.2	–	–	–	–	–	69	1	69	54.40	613.34	938	831	69	335	68	54.42	399.06	450	755
T1.3	–	–	–	–	–	67	840	67	59.24	747.63	1058	532	68	334	67	58.93	279.83	390	498
T1.4	–	–	–	–	–	75	4	74	57.51	912.69	879	958	75	171	74	57.75	486.04	480	910
T1.5	–	–	–	–	–	63	239	63	51.37	873.18	1087	704	64	17	64	51.20	383.62	480	718
T2.1	–	–	–	–	–	51	559	51	40.22	991.70	904	1001	52	36	50	40.04	516.20	480	872
T2.2	–	–	–	–	–	55	408	55	39.80	1161.20	861	1334	56	16	56	39.37	496.70	360	1351
T2.3	–	–	–	–	–	54	135	53	39.27	1228.62	986	1221	55	60	55	39.29	600.98	480	1265
T2.4	–	–	–	–	–	53	444	51	39.91	1120.65	910	1002	55	79	54	39.91	689.06	570	1135
T2.5	–	–	–	–	–	55	21	53	39.26	1138.79	860	1243	54	281	54	39.06	623.07	480	1232
T3.1	–	–	–	–	–	44	36	42	30.37	1591.86	974	1600	44	15	44	30.10	633.35	390	1683
T3.2	–	–	–	–	–	48	623	48	32.80	1969.72	1077	1905	49	4	49	31.88	793.57	420	2007
T3.3	–	–	–	–	–	44	789	43	29.72	1848.07	1096	1656	45	1	45	29.61	802.08	480	1755
T3.4	–	–	–	–	–	44	499	44	31.60	1758.73	1074	1683	45	61	44	31.29	713.84	420	1693
T3.5	–	–	–	–	–	45	117	44	31.06	1902.47	1132	1686	45	14	45	30.76	692.85	420	1690

Table 5
Average percentage deviation from optimal

Problem set	LH	MLH	SH
4	0.058	0.020	0.041
5	0.179	0.212	0.000
6	0.559	0.820	0.411
A	0.818	0.898	0.659
B	0.806	1.046	1.046
C	1.931	0.890	1.482
D	2.746	0.934	2.090
Overall mean	0.815	0.561	0.641

Table 6
Average number of subgradient iteration

Problem set	LH	MLH	SH
4	574.3	505.2	232.9
5	622.1	501.4	272.8
6	775.8	819.4	426.0
A	735.0	779.8	402.0
B	801.2	905.2	438.0
C	910.2	877.2	450.0
D	852.0	974.8	414.0
T1	–	998.8	450.0
T2	–	904.2	474.0
T3	–	1070.6	426.0

Table 7
Average computational times (sec.)

Problem set	MLH	SH	% [a]
4	17.18	13.24	22.9
5	18.77	16.47	12.3
6	44.09	28.02	36.4
A	58.44	42.68	27.0
B	116.94	60.13	48.6
C	111.44	75.10	32.6
D	186.71	88.80	52.4
T1	752.50	384.17	49.0
T2	1128.19	585.20	48.1
T3	1814.17	727.40	59.9
Total time	21421.90	10254.60	52.1

[a] $100 \times \{(\text{MLH} - \text{SH})/\text{MLH}\}$.

5. Conclusions

A new surrogate heuristic for large scale set covering problems, called SH, has been developed. SH combines an efficient data structure, good reduction tests, an efficient method to solve the continuous surrogate problem, an adequate step size control, and especially the use of intermediate continuous surrogate problems.

The stability obtained in the sequences of intermediate surrogate values appears to be a nice substitute to the Lagrangian ones, preserving all their strong results.

Algorithm SH seems promising due to the favorable comparisons with the dual heuristic of Beasley [8], one of the best known heuristics for the set covering problem.

Acknowledgments

We thank Dr. J.E. Beasley who has made available the test problem sets 4, 5, 6, A, B, C and D used in the computational tests.

The first author acknowledges the Conselho Nacional de Pesquisas, CNPq (proc. 500756/90-2), and the Fundação de Amparo à Pesquisa do Estado de São Paulo, FAPESP (proc. 89/3725-5), for partial financial support.

The second author acknowledges the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior, CAPES (proc. CAP 33150028).

References

[1] Allen, E., Helgason, R., Kennington, J., and Shetty, A., "A generalization of Polyak's convergence results for subgradient optimization", *Mathematical Programming* 37 (1987) 309–317.

[2] Baker, E.K., Bodin, L.D., Finnegan, W.F., and Ponder, R.J., "Efficient heuristic solutions to an airline crew scheduling problem", *AIIE Transactions* 11 (1979) 79–85.

[3] Balas, E., and Ho, A., "Set covering algorithms using cutting planes, heuristics, and subgradient optimization: A computational study", *Mathematical Programming* 12, (1980) 37–60.

[4] Balas, E., and Padberg, M.W., "Set partitioning – A survey", in: N. Christofides (ed.), *Combinatorial Optimization*, Wiley, New York, 1979.

[5] Balas, E., and Zemel, E., "An algorithm for large zero-one knapsack problems", *Operations Research* 28/5 (1980) 1130–1154.

[6] Bazaraa, M.S., and Sherali, H.D., "On the choice of step size in subgradient optimization", *European Journal of Operational Research* 7 (1981) 380–388.

[7] Beasley, J.E., "An algorithm for the set covering problem", *European Journal of Operational Research* 31 (1987) 85–93.

[8] Beasley, J.E., "A Lagrangian heuristic for the set covering problem", *Naval Research Logistics* 37 (1990) 151–164.

[9] Bowers, J., "Network analysis on a micro", *Journal of the Operational Research Society* 36 (1985) 609–612.

[10] Clarke, F.H., "Generalized gradients and applications", *Transactions of the American Mathematical Society* 205 (1975) 247–262.

[11] Christofides, N., and Korman, S., "A computational survey of methods for the set covering problem", *Management Science* 21 (1975) 591–599.

[12] Dantzig, G.B., "Discrete-variable extremum problems", *Operations Research* 5 (1957) 266–277.

[13] Day, R.H., "On optimal extracting from a multiple file data storage system: An application of integer programming", *Operations Research* 13/3 (1965) 482–494.

[14] Dudzinski, K., and Walukiewicz, S., "Exact methods for the knapsack problem and its generalizations", *European Journal of Operational Research* 28 (1987) 3–21.

[15] Dyer, M.F., "Calculating surrogate constraints", *Mathematical Programming* 19 (1980) 225–278.

[16] Fayard, D., and Plateau, G., "An algorithm for the solution of the 0–1 knapsack problem", *Computing* 28 (1982) 269–287.

[17] Fisher, M.L., and Kedia, D., "Optimal solution of set covering/partitioning problems using dual heuristics", *Management Science* 36/6 (1990) 674–688.

[18] Foster, B.A., and Ryan, D.M., "An integer programming approach to the vehicle scheduling problem", *Operational Research Quarterly* 27 (1976) 367–384.

[19] Fukushima, M., "A descent algorithm for nonsmooth convex optimization", *Mathematical Programming* 30 (1984) 163–175.

[20] Garey, M.R., and Johnson, D.S., *Computers and Intractability: A Guide to the Theory of NP-completeness*, Freeman, San Francisco, CA, 1979.

[21] Garfinkel, R.S, Neeb, A.W., and Rao, M.R., "The m-center problem: Bottleneck facility location", Working Paper No. 7414, Graduate School of Management, University of Rochester, Rochester, NY, 1974.

[22] Garfinkel, R.S., and Nemhauser, G.L., "The set partitioning problem: Set covering with equality constraints", *Operations Research* 17 (1969) 848–856.

[23] Gavish, B., and Pirkul, H., "Efficient algorithms for solving multiconstraint zero-one knapsack problems to optimality", *Mathematical Programming* 31 (1985) 78–105.

[24] Goffin, J.L., "On convergence rates of subgradient optimization methods", *Mathematical Programming* 13 (1977) 329–347.

[25] Golden, B.L., Assad, A.A., and Wasil, E.A. (Guest editors, Special Issue), "Microcomputers in Operations Research", *Computer & Operations Research* 13 (21/3) (1986).

[26] Greenberg, H.J., "The generalized penalty function surrogate model", *Operations Research* 21 (1973) 162–178.
[27] Harrison, T.P., "Micro versus mainframe performance for a selected class of mathematical programming problems", *Interfaces* 15 (1985) 14–19.
[28] Held, M., Wolfe, P., and Crowder, H.P., "Validation of subgradient optimization", *Mathematical Programming* 6 (1974) 62–88.
[29] John, C.G., and Kochenberger, G.A., "Using surrogate constraints in a Lagrangian relaxation approach to set-covering problems", *Journal of the Operational Research Society* 39/7 (1988) 681–685.
[30] Karwan, M.H., and Rardin, R.L., "Surrogate dual multiplier search procedures in integer programming", *Operations Research* 32 (1984) 562–569.
[31] Kiwiel, K.C., "An aggregate subgradient method for nonsmooth convex minimization", *Mathematical Programming* 21 (1983) 320–341.
[32] Lorena, L.A.N., and Plateau, G., "A monotone decreasing algorithm for the 0–1 multiknapsack dual problem", Rapport de recherche L.I.P.N. 89-1, Université Paris-Nord, France, 1988.
[33] Marsten, R.E., An algorithm for large set partitioning problems", *Management Science* 20 (1974) 774–787.
[34] Marsten, R.E., and Shepardson, F., "Exact solutions of crew scheduling problems using the set partitioning problem: Recent successful applications", *Networks* 11 (1981) 165–177.
[35] McKay, A.C., "Linear programming applications on microcomputers", *Journal of the Operational Research Society* 36 (1985) 633–636.
[36] Murdoch, J., "Forecasting and inventory control on micros", *Journal of the Operational Research Society* 36 (1985) 607–608.
[37] Nemhauser, K., and Wolsey, L.A., *Integer and Combinatorial Optimization*, Wiley, New York, 1988.
[38] Pirkul, H., "A heuristic solution procedure for the multiconstraint zero–one knapsack problem", *Naval Research Logistics* 34 (1987) 161–172.
[39] Polyak, B.T., "A general method of solving extremum problems", *Soviet Mathematics Doklady* 8 (1967) 593–597.
[40] Polyak, B.T., "Maximization of unsmooth functionals", *U.S.S.R. Computational Mathematics and Mathematical Physics* 9/3 (1969) 14–26.
[41] Quine, W.V., "A way to simplify truth functions", *The American Mathematical Monthly* 62 (1955) 627–631.
[42] Revelle, C., Marks, D., and Liebman, S.C., "An analysis of private and public section facilities location models", *Management Science* 16/12 (1970) 692–707.
[43] Salkin, H.M., and Koncal, R.D., "Set covering by an all-integer algorithm computational experience", *Journal of the ACM* 20 (1973) 189–193.
[44] Salverson, M.E., "The assembly line balancing problem", *Journal of Industrial Engineering* 6 (1955) 18–25.
[45] Sarin, S., Karwan, M.H., and Rardin, R.L., "A new surrogate dual search procedure in integer programming", *Naval Research Logistics* 34 (1984) 431–450.
[46] Schramm, H., "A combination of bundle and trust region methods in nondifferentiable nonconvex optimization", Research Report 149, Mathematisches Institut, Universität Bayreuth, 1989.
[47] Shor, N.Z., "On the structure of algorithms for the numerical solution of optimal planning and design problems", Dissertation, Cybernetics Institute, Academy of Science, USSR, 1964.
[48] Shor, N.Z., *Minimization Methods for Non-differentiable Functions*, Springer-Verlag, Berlin, 1985.
[49] Tremblay, J.P., and Sorenson, P.G., *An Introduction to Data Structures with Applications*, McGraw-Hill, Singapore, 1984.
[50] Vasko, F.J., and Wilson, G.R., "An efficient heuristic for large set covering problems", *Naval Research Logistics Quarterly* 31 (1984) 163–171.
[51] Vasko, F.J., and Wolf, F.E., "Solving large set covering problems on a personal computer", *Computers & Operations Research* 15/2 (1988) 115–121.
[52] Walker, W., "Application of the set covering problem to the assignment of ladder trucks to fire houses", *Operations Research* 22 (1974) 275–277.
[53] Wheller, F.P., "A starter kit for micro-based LP solvers", *Journal of the Operational Research Society* 36 (1985) 637–642.